

AI 编程 Token 经济学：理解消耗机制，让同样的额度多干 3 倍活



鹅厂架构师
已认证机构号

7 人赞同了该文章

月底了，大家的 Token 是不是又不够用了？来看看它到底是怎么花掉的

作者：鹅厂架构师 alvywang

导读

全文五章，从“钱花在哪”到“怎么少花”：

- 一、你的一句话背后：一次 API 调用到底发了什么
- 二、缓存机制⁺：为什么长对话反而比新窗口便宜 5 倍
- 三、四类配置的加载机制与成本控制
- 四、Sub-Agent：不省钱，但能防膨胀
- 五、优化策略：配置侧 + 对话侧
- 附：Token 流向全景图
- 附：常见误区纠正

一、你的一句话背后：一次 API 调用到底发了什么

1.1 Token 不是字数

Token 是模型处理文本的最小单元。不同模型的 `tokenizer`⁺ 略有差异，以 Claude⁺ / GPT 系列为例的粗略换算：

语言	换算关系	说明
英文	1 token $\approx$ 4 个字符 $\approx$ 0.75 个单词	"hello" = 1 token, "implementation" = 1 token
中文	1 个汉字 $\approx$ 1~2 tokens	常用字约 1 token, 生僻字/组合约 2 tokens
代码	差异较大	关键词/变量名/符号各占 1 token, 长变量名 ⁺ 可能拆成多个

一段 200 行的 Python 脚本大约 2,000~4,000 tokens；一段 1000 字的中文约 1,200~1,800 tokens。

1.2 一次调用的完整结构

每当你在 AI 编程工具里敲一句话、或 AI 调用一个工具后准备回复，都是一次完整的用。每次调用发送的内容结构如下：

① System Prompt（系统指令）

~8k-20k tokens

赞同 7

添加评论

11

喜欢

分享

申请转载

✦

关于作者



鹅厂架构师

走进腾讯架构师的世界

已认证机构号

回答

20

文章

464

关注者

25,165

+ 关注

发私信

大家都在搜

换一换

人民日报评第一学历歧视 370 万

热

表演艺术家游本昌去世 355 万

新

任正非重申华为不造车 340 万

热

曝豆包开始收缩对话团队 338 万

新

看山今日一签 338 万

活

亚运男足 想格斗可以别的... 338 万

热

腾讯「龙虾」将正式停止运... 338 万

新

多所高校取消吸烟学生学费... 337 万

热

车企集体和宁德提分手 335 万

热

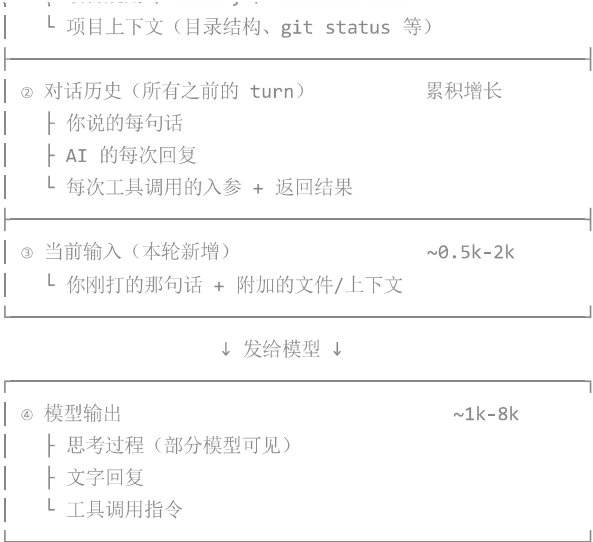
张家齐看到妈妈出丑后冷笑 323 万

热

登录即可查看 超 5 亿 专业优质内容

超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。

立即登录/注册



关键认知：你打的那句话（③）通常只占总输入的 1%~5%。真正的大头是 ① 和 ②。

二、缓存机制：为什么长对话反而比新窗口便宜 5 倍

2.1 没有缓存会怎样——O(N²) 的恐怖增长

如果每次 API 调用都要从头全价计算所有输入 Token，成本会是这样：

假设：System Prompt 15k，每轮新增 2k（你的话 + AI 回复 + 工具结果）

Turn 1: 15k + 2k = 17k 全价

Turn 2: 15k + 4k = 19k 全价 （包含 Turn 1 的历史）

Turn 3: 15k + 6k = 21k 全价

...

Turn 10: 15k + 20k = 35k 全价

10 轮总计输入：~260k tokens（全价） ← 二次增长 O(N²)

每一轮都把之前的所有内容重发一遍，**重复付费**。这就是为什么早期 API 用户感觉"聊几轮就烧光了"。

2.2 KV 缓存的原理

大模型用 **Transformer 注意力机制** 处理文本，其中历史 Token 的 Key 和 Value 张量算完就不变了。把它们存起来，下次直接读取，不用重算。这就是 **KV Cache**。

2.3 三档价格：写入、命中、全价

以 Claude 为例（[官方文档](#)），Token 有三档价格：

Token 类型	价格倍率	触发条件
缓存写入 (cache write)	基础价 × 1.25	首次出现的前缀，写入
缓存命中 (cache read)	基础价 × 0.1	再次出现的前缀，命中
全价输入 (uncached)	基础价 × 1.0	全新的 Token，未命中

×

登录即可查看 **超 5 亿** 专业优质内容

超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。

模型	基础输入	缓存与入	缓存读取	输出
Claude Opus 4	\$5	\$6.25	\$0.50	\$25
Claude Sonnet 4	\$3	\$3.75	\$0.30	\$15
Claude Haiku 4.5+	\$1	\$1.25	\$0.10	\$5

缓存读取只要基础输入价的 1/10。对 Opus 来说，缓存命中的输入价格（）甚至比的基础输入价（）甚至比的基础输入价（1/M）还便宜。

注意：Claude Code+ 订阅产品（Pro/Max）与直接调用 API 的缓存机制有所不同。Claude Code 在产品层面为订阅用户实现了扩展缓存有效期（源码中为 1 小时），这是产品侧的优化，不等同于 API 的付费选项。

2.4 有缓存后：同样 10 轮，省 76%

Turn 1: [System 15k: 写入×1.25] + [新增 2k: 全价]
等价 = 15k×1.25 + 2k = 20.75k ← 首次略贵

Turn 2: [System 15k: 命中×0.1] + [T1 历史 2k: 命中×0.1] + [新增 2k: 全价]
等价 = 15k×0.1 + 2k×0.1 + 2k = 3.7k ← 骤降！

Turn 3: 等价 = 15k×0.1 + 4k×0.1 + 2k = 3.9k
Turn 5: 等价 = 15k×0.1 + 8k×0.1 + 2k = 4.3k
Turn 10: 等价 = 15k×0.1 + 18k×0.1 + 2k = 5.3k

10 轮总计: ~62k 等价 tokens

	无缓存	有缓存
10 轮总输入 token	~260k 全价	~260k (但大部分 ×0.1)
等价成本	260k	~62k
节省	—	76%
增长曲线+	O(N²)	近似 O(N)

越长的对话，缓存覆盖率越高，每轮**边际成本+**越低。这就是为什么"一个 Session 持续对话"远比"频繁开新 Session"划算。

2.5 缓存只能从头匹配——中间断了全废

缓存只能从头开始匹配。想象你的 Prompt 是一条链：

[System 指令] → [工具定义] → [Rules/Memory] → [msg1] → [msg2] → [msg3]

如果链条中间任何一环变了，该环及之后的所有内容全部失效：

最省：只追加新消息
[✅ System] [✅ Tools] [✅ Rules] [✅ msg1] [✅ msg2] [新 msg3]
→ 只有 msg3 全价

中等：修改了 Rules/Memory
[✅ System] [✅ Tools] [❌ Rules] [❌ msg1] [❌

×

登录即可查看 超5亿 专业优质内容

超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。



[X System] [X Tools] [X Rules] [X msg1] [X msg2] [X msg3]
→ 不同模型的 KV 张量不互通，从头全价

2.6 缓存有效期与保活

API 层面（直接调用 Anthropic+ API）：默认 5 分钟过期，付费选项可延长至 1 小时。

Claude Code 产品层面（源码逻辑，claude.ts:408-413）：

- 普通用户：5 分钟
- Pro/Max 订阅且未超额：延长至 1 小时

刷新机制：每次缓存命中都会重置过期计时。所以只要在过期前发一次匹配前缀的请求，缓存就能无限续命。

实操影响：午饭 1.5 小时回来 → 缓存过期 → 冷启动+。可以设置定时脚本+每 55 分钟发一句简单的话（如"ok"）来保活。

三、四类配置的加载机制与成本控制

System Prompt 是每轮必带的"固定税"，但不是所有配置都会常驻在 System Prompt 里。
AI 编程工具+提供了四类可配置项，它们的复杂度递增、加载策略各异。理解它们的区别，既是控制成本的关键，也决定了你应该把什么内容放在哪一层。

3.1 全景：从全量常驻到完全按需



×

登录即可查看 超5亿 专业优质内容

超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。



Memory 的制定在所有记忆条目主里添加后注入 System Prompt，又有体系、又有用途。

- 你有 20 条 Memory 记录：
- "缩进统一用 4 空格"

- "函数命名用 snake_case"

- "部署脚本在 scripts/ 目录下"

- "文档格式用 Markdown"

- ...

→ 相关（写代码时）

→ 相关（写代码时）

→ 不相关（大部分时候）

→ 不相关（大部分时候）

全部 20 条 → ~800 tokens → 每轮都带 → 但大部分场景只有 2-3 条相关

直接 Token 成本很低（几百 tokens），但有两个隐性影响：

1. 位置效应+：Memory 在 System Prompt 中，改动任意一条都会打断缓存链
2. 噪声效应+：无关的 Memory 占据模型的注意力权重，可能干扰输出质量

什么时候用 Memory：一句话能说清的偏好或约定，比如"SQL 中不等于用 <> "、"字符串格式化用 .format()"。不要把复杂的规范或工作流放在 Memory 里。

3.3 第二层：Rules——高频常驻，低频触发

Rules（.mdc 文件）比 Memory 更结构化，支持三种加载模式：

模式	加载时机	Token 影响
常驻（always）	每轮都在 System Prompt 中	固定增加系统税，但缓存后 ×0.1
触发式（requested/auto）	AI 根据用户消息的关键词/语义判断是否加载	不触发时 0 成本，触发时动态注入
手动（manual）	用户显式 @ 或调用时才加载	完全按需

实际案例：

- code-style-guide.mdc

→ always

→ 每轮常驻 ~2k tokens（高频，几乎每个任务都用
- api-gen-template.mdc

→ triggered

→ 提到"生成 API"时才注入 ~1.5k tokens
- migration-check.mdc

→ triggered

→ 提到"数据迁移+"时才注入 ~1k tokens

什么时候用 Rules：需要几百到几千字描述的编码规范、格式约束、特定场景模板。高频规则设为常驻，低频规则设为触发式。和 Memory 的区别在于：Rules 可以按需加载，Memory 只能全量常驻。

3.4 第三层：Skills——加载前零开销，加载后回不去

Skills 是最重的配置单元，通常包含：

- SKILL.md：技能指令文档（可达 5k-15k tokens）
- references/：参考文档
- scripts/：可执行脚本

阶段	是否在上下文	Token 消耗
未加载	仅 description 在可用列表中（~50 tokens/skill）	极低
用户触发或 AI 判断需要 → use_skill	SKILL.md 全文注入到对话	一次性 5k-15k
加载后的后续轮次	留在对话历史中	

登录即可查看 超 5 亿 专业优质内容

超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。



- 加载前：零开销
- 加载后：首轮全价，后续缓存
- 不会影响 System Prompt 的缓存链
- 但一旦加载就无法卸载，5k-15k tokens 永久留在历史中

什么时候用 Skills：需要完整工作流的复杂任务，比如"数仓+开发全流程（建表→改表→取数→审查）"。和 Rules 的区别在于：Skills 承载的是带脚本、带参考文档的完整工作流，而非单条规范。

3.5 第四层：MCP 工具——Schema 常驻，结果永久留存

MCP (Model Context Protocol) 工具是外部能力的接入点，比如浏览器控制、数据库查询、部署服务等。

它的成本有两部分：

1. **工具定义 (JSON Schema)：**每个工具的 Schema 约 200-500 tokens，**常驻在 System Prompt 中**。10 个闲置工具 = 2k-5k tokens 的固定税
2. **工具调用结果：**每次调用的返回结果**永久留在对话历史中**

工具操作	典型返回大小	后续每轮重发成本（缓存 ×0.1）
读一个 200 行文件	3k-5k tokens	300-500 等价/轮
搜索代码（10 条结果）	2k-4k tokens	200-400 等价/轮
执行命令（长输出）	1k-10k tokens	100-1,000 等价/轮

单看 ×0.1 不多，但工具调用频繁时会快速累积。一个复杂任务调用 20 次工具，历史里就积累了 40k-100k 的工具结果。

什么时候该关注 MCP 成本：当你发现 Session 后期每轮都变慢、变贵时，大概率是工具调用结果把对话历史撑大了。解决方案见第五章。

3.6 选择指南：什么内容放在哪一层

内容类型	推荐层级	原因
"变量命名用 snake_case"	Memory	一句话偏好，全量常驻成本低
代码格式规范（缩进、命名、注释规则）	Rules（常驻）	几千字的结构化规范，高频使用
特定场景的代码生成模板	Rules（触发式）	低频场景，不需要每轮都带
完整开发流程（需求分析→编码→测试→部署）	Skills	完整工作流 + 参考文档 + 脚本
浏览器自动化、数据库查询	MCP 工具	需要外部能力

3.7 缓存链全貌：断在哪里，后面全废

把配置项按它们在 Prompt 中的**实际排列顺序**画出来：

缓存链条（断在哪里，后面全废）：

[核心指令 ~4k] → [工具定义 ~6k] → [Rules(always) ~2k]

登录即可查看 超5亿 专业优质内容

超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。



缓存是前缀匹配——断在越前面，后面废掉的越多。



四、Sub-Agent：不省钱，但能防膨胀

4.1 什么是 Sub-Agent

复杂任务中，主 Agent 可能启动独立的子 Agent 来完成特定工作（如 code-explorer 搜索代码、reviewer 审查代码）。

4.2 Sub-Agent 无法复用主 Agent 的缓存

Sub-Agent 几乎不能复用主 Agent 的缓存。原因：

- 1. 工具集不同 → 工具 Schema 不同 → 缓存前缀不同
- 2. 消息历史独立 → 对话内容完全不同
- 3. 可能用不同模型 → KV 张量不互通

主 Agent (Turn 8, 上下文 40k):
[System 15k: 命中×0.1] + [历史 23k: 命中×0.1] + [新增 2k: 全价]
等价成本 = 1.5k + 2.3k + 2k = 5.8k ← 大部分缓存

Sub-Agent (Turn 1, 全新上下文):
[System 10k: 写入×1.25] + [传入 prompt 3k: 全价]
等价成本 = 12.5k + 3k = 15.5k ← 冷启动全价！

4.3 什么时候值得用 Sub-Agent

场景	推荐方式	原因
简单的文件搜索/读取	主 Agent 直接做	缓存好，边际成本低
大范围代码探索 (20+ 文件)	Sub-Agent	避免大量工具结果污染主上下文
代码审查	Sub-Agent	需要独立视角，且审查结果不应膨胀主上下文
需要用更便宜模型的任务	Sub-Agent	子任务用 Sonnet/Haiku, 省模型费用

Sub-Agent 的核心价值不是省 Token，而是防止主上下文被工具结果无限膨胀。

五、优化策略：配置侧 + 对话侧

前面四章讲的是“钱花在哪”，这一章讲**怎么少花**。分两个维度：配置侧（Session 开好）和对话侧（Session 进行中注意）。

5.1 配置侧：Session 开始前配好，中途别动

登录即可查看 超5亿 专业优质内容

超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。



Rules

策略	做法	节省效果
精减常驻 Rules	只把高频规则设为 alwaysApply: true，低频规则用触发式	每条规则省 1k-3k tokens/轮系统税
合并同类规则	把 5 个小规则合并成 1 个大规则，减少元数据开销	减少 Rules 描述注入量
精炼规则内容	去掉冗余示例，保留核心约束	每条省 30-50%
善用触发式关键词	在 description 里写精准的触发关键词，避免误触发	减少不必要的注入

- ✗

所有 5 条规则都 alwaysApply: true
→ 系统税 = 5k-15k tokens（其中大部分场景只用到 1 条）
- ✓

1 条高频规则 alwaysApply: true + 4 条触发式
→ 系统税 = 1k-3k tokens（其余按需加载）
→ 节省 60-80% 的 Rules 系统税

Skills

策略	做法	节省效果
不要预加载	让 AI 根据任务自动判断是否需要加载	避免 5k-15k 的无效注入
精简 SKILL.md	把参考文档放 references/ 而非写进主文件	加载时只注入核心指令
写好 description	精确的 description 让 AI 更准确地判断何时加载	减少误加载

Memory

策略	做法	节省效果
定期清理	删除过时或冗余的条目	减少噪声 token
合并同类记忆	多条相似偏好合并为一条	减少条目数
写法精炼	每条控制在 1-2 句话	总量控制在 500 tokens 以内

MCP 工具

策略	做法	节省效果
禁用闲置工具	每个 Schema 约 200-500 tokens，10 个闲置 = 2k-5k 浪费	减少 Schema 大小
按项目配置	不同项目用不同的工具配置，而非全局开启	减少 Schema 大小

项目文档（CODEBUDDY.md）

策略	做法	节省效果
精炼内容	控制在 2k-5k tokens，用指针引用其他文档	减少系统税
用引用代替内联	"详见 .codebuddy/skills/xxx/SKII	

×

登录即可查看 超5亿 专业优质内容

超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。



优化前（所有 Rules 常驻 + 10 个 MCP + 30 条 Memory + 大 CODEBUDDY.md）：
System Prompt ≈ 25k tokens → 后续每轮 = 25k × 0.1 = 2.5k 等价

优化后（1 条常驻 Rule + 精简 MCP + 精炼 Memory + 精炼 CODEBUDDY.md）：
System Prompt ≈ 12k tokens → 后续每轮 = 12k × 0.1 = 1.2k 等价

每轮节省 1.3k 等价，100 轮节省 ~130k 等价 tokens

5.2 对话侧：Session 进行中的行为习惯

保护缓存（最重要）

#	原则	说明
1	一个 Session 干到底	新 Session = 冷启动全价 (~15-25k)，老 Session 继续只付增量，省 5 倍
2	别频繁切模型	Opus → Sonnet → Opus = 两次冷启动
3	超过一小时没说话？缓存已过期	缓存有有效期，超时就过期。持续工作时保持对话活跃即可

减少无效 Token

#	原则	说明
4	一次说清需求	"建一张日表，字段有 A/B/C，分区 ds" 比分 3 轮说省 60% 轮次
5	先描述再贴代码	"第 15 行 join 条件写反了" 比直接贴 500 行代码高效
6	别反复问"要不要继续"	每轮对话都有成本，AI 能自主完成的就让它做完
7	复杂任务写 plan 文件	多 Session 任务把方案写到文件里，下次直接读取，跨 Session 省 80%

附：Token 流向全景图



登录即可查看 超5亿 专业优质内容

超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。





记住这三个数字：

- **System Prompt 是"固定税"**：~12-25k tokens，但缓存后只要 ~1.2-2.5k 等价（配置侧优化可压低基数）
- **对话历史是"累积税"**：线性增长，但缓存后只付 1/10
- **你打的字是"零头"**：真正的成本在你看不见的地方

附：常见误区纠正

误区	真相
"我就打了一行字怎么这么贵"	你的一行字只占 1%，其余 99% 是系统指令和对话历史
"长对话越来越贵，应该经常开新窗口"	恰好相反——长对话缓存命中率（×0.1），新窗口才是最贵的操作（全价冷启动）
"Sub-Agent 能省 Token"	Sub-Agent 每次冷启动无缓存，等价成本可能更高。它的价值是隔离上下文
"Memory 很耗 Token"	Memory 本身只有几百 tokens，但好的 Memory 能省掉数轮重复交代
"/compact 能省钱"	Compact 压缩历史 → 缓存断裂。除非上下文接近窗口上限，否则别用
"Rules 越多越好"	常驻 Rules 增加系统税。低频规则应设为触发式，按需加载
"Skills 加载了不用也没关系"	Skills 的 SKILL.md 一旦加载就留在历史中，5k-15k tokens 无法撤回
"配好环境后随时可以改"	改 Rules/Memory/CODEBUDDY.md/MCP 工具都会打断缓存链，应在 Session 开始前配好

一句话总结：AI 编程的 Token 成本不在于你说了多少，而在于两件事——配置侧：系统税（Rules 按需加载、禁用无用工具、精炼 Memory）；对话侧：保护缓存（一个

登录即可查看 超5亿 专业优质内容

超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。



编辑于 2026-04-28 10:57 · 广东



维达 PRO 超厚韧厨房纸 厨房油污一抹干净

维达 PRO 超厚韧厨房纸，3层加厚，湿用仍强韧，10倍吸油锁水，备餐清洁一张搞定。 [查看详情](#)

维达 的广告

 未登录用户



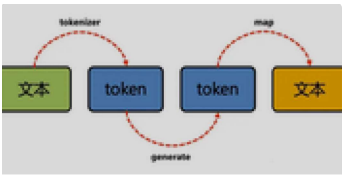
还没有评论，发表第一个评论吧

推荐阅读

大语言模型中的 Token 计算与费用控制

什么是 Token? Token 是文本的基本单位，可以是单词、子词、标点符号或符号。每当您向 API 发送请求时，文本会被分解成 tokens，以便模型理解并生成响应。对中文而言，1 个汉字通常算作 1 ...

eden 发表于知界 | ...



【大模型】什么是Token?有啥作用和意义?

智驾工程笔记

大模型核心概念科普：Token、上下文长度、最大输出，一次...

Token 是什么 token 是大模型（LLM）用来表示自然语言文本的基本单位，可以直观的理解为“字”或“词”。通常 1 个中文词语、1 个英文单词、1 个数字或 1 个符号计为 1 个 token 一般情况... 小盒子



大模型的 Token：AI 世界的“货币单位”，那么一个Toke...

北方的郎 发表于北方的郎